

Project 3: Wireshark Network Traffic Analysis

My Findings:

Ok, So I started this one off by opening up chrome and wireshark and finding my connection type (wireless). I then started the packet capture and opened Youtube, Facebook, and Gmail. Let's see what that looks like!

No.	Time	Source	Destination	Protocol	Length	Info
23895	28.937475500	2607:f8b0:4002:c06:...	2600:1700:3490:7550...	TLSv1.3	696	Application Data
23896	28.937475800	2607:f8b0:4002:c06:...	2600:1700:3490:7550...	TLSv1.3	231	Application Data
23897	28.937498500	2600:1700:3490:7550...	2607:f8b0:4002:c06:...	TCP	74	60924 → 443 [ACK] Seq=426361 Ack=5664544 Win=1048320
23898	28.937662000	2600:1700:3490:7550...	2607:f8b0:4002:c06:...	TLSv1.3	109	Application Data
23899	28.938684500	2607:f8b0:4002:c06:...	2600:1700:3490:7550...	TLSv1.3	348	Application Data
23900	28.938684900	2607:f8b0:4002:c06:...	2600:1700:3490:7550...	TLSv1.3	113	Application Data
23901	28.938702000	2600:1700:3490:7550...	2607:f8b0:4002:c06:...	TCP	74	60924 → 443 [ACK] Seq=426396 Ack=5664857 Win=1048064
23902	28.939089500	2600:1700:3490:7550...	2607:f8b0:4002:c06:...	TLSv1.3	113	Application Data
23903	28.941308500	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	94	Standard query 0x989b HTTPS www.google.com
23904	28.941422500	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	94	Standard query 0x149b AAAA www.google.com
23905	28.941509200	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	94	Standard query 0x97fb A www.google.com
23906	28.943604600	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	123	Standard query response 0x989b HTTPS www.google.com
23907	28.943604900	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	226	Standard query response 0x97fb A www.google.com A 14
23908	28.951403800	2607:f8b0:4002:c06:...	2600:1700:3490:7550...	TCP	74	443 → 60924 [ACK] Seq=5664857 Ack=426435 Win=416512
23909	28.964415300	2600:1700:3490:7550...	2600:1700:3490:7550...	DNS	322	Standard query response 0x149b AAAA www.google.com A

Frame 1: Packet, 107 bytes on wire (856 bits), 107 bytes captured
Ethernet II, Src: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61), Dst:
Internet Protocol Version 4, Src: 162.159.134.234, Dst: 192.168.1.
Transmission Control Protocol, Src Port: 443, Dst Port: 55372, Seq
Transport Layer Security

So right off the bat, it looks a bit overwhelming, there is a lot of information here. Over 23 thousand packets were captured in less than a minute! I am going to filter this down a bit so that I can look more closely at specific events and protocol types.

The first protocol I want to take a look at is DNS.

```
Frame 53: Packet, 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface \Device\NPF_{C88
Ethernet II, Src: Intel_ec:9d:d9 (e4:60:17:ec:9d:d9), Dst: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61)
Internet Protocol Version 6, Src: 2600:1700:3490:7550:4c1f:5809:3f33:6a53, Dst: 2600:1700:3490:7550::1
User Datagram Protocol, Src Port: 59986, Dst Port: 53
Domain Name System (query)
```

Taking a look at the first entry, I can see that this DNS query used UDP, had a source port of 59986 and a destination port of 53. That checks out, DNS is on port 53. Port 59986, however, is what is known as an ephemeral port. It is just a temporary port for communication. I see we are using IPV6, which is more common as we transition away from IPV4.

Protocol	Length	Info
DNS	94	Standard query 0xbc76 AAAA www.google.com
DNS	94	Standard query 0xe85f A www.google.com
DNS	123	Standard query response 0x773c HTTPS www.google.com HTTP
DNS	322	Standard query response 0xbc76 AAAA www.google.com AAAA
DNS	94	Standard query 0x42df HTTPS t2.gstatic.com
DNS	94	Standard query 0xb018 AAAA t2.gstatic.com
DNS	94	Standard query 0xefcc A t2.gstatic.com

This next image shows a few other bits of useful networking information. Here I can see that a query was sent to google.com for a “AAAA” record. This is a way for the server to match a domain name to an IPV6 address.

No.	Time	Source	Destination	Protocol	Length	Info
54	2.524155900	2600:1700:3490...	2600:1700:3490:75...	DNS	94	Standard query 0xbc76 AAAA www.google.com
55	2.524215200	2600:1700:3490...	2600:1700:3490:75...	DNS	94	Standard query 0xe85f A www.google.com
56	2.526740500	2600:1700:3490...	2600:1700:3490:75...	DNS	123	Standard query response 0x773c HTTPS www.google.com HTTP
57	2.527456400	2600:1700:3490...	2600:1700:3490:75...	DNS	322	Standard query response 0xbc76 AAAA www.google.com AAAA
58	2.536550900	2600:1700:3490...	2600:1700:3490:75...	DNS	94	Standard query 0x42df HTTPS t2.gstatic.com
59	2.536672600	2600:1700:3490...	2600:1700:3490:75...	DNS	94	Standard query 0xb018 AAAA t2.gstatic.com
60	2.536742900	2600:1700:3490...	2600:1700:3490:75...	DNS	94	Standard query 0xefcc A t2.gstatic.com

Frame 57: Packet, 322 bytes on wire (2576 bits), 322 bytes captured (2576 bits) on interface \Device\NPF_{C88E6B3E-ACC5-4A9...}

Ethernet II, Src: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61), Dst: Intel_ec:9d:d9 (e4:60:17:ec:9d:d9)

802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 0

Internet Protocol Version 6, Src: 2600:1700:3490:7550::1, Dst: 2600:1700:3490:7550:4c1f:5809:3f33:6a53

User Datagram Protocol, Src Port: 53, Dst Port: 56996

Domain Name System (response)

This image shows me the resolution of that record. I can also see in the details that IEEE 802.1q, which is VLAN tagging, was used to direct this traffic. Nokia and Intel both show on here as well, however I know that is simply based on the MAC address. The first half of which is always the manufacturer/vendor information. So that means that one of the devices that was in the communication path was Intel and one was Nokia.

Next, I will take a look at TCP protocols.

27	2.263122700	2600:1700:3490...	2606:4700:4403::a...	TLSv1.2	164	Application Data
30	2.276874600	2606:4700:4403...	2600:1700:3490:75...	TCP	74	443 → 52075 [ACK] Seq=80 Ack=181 Win=17 Len=0
35	2.334768500	2606:4700:4403...	2600:1700:3490:75...	TLSv1.2	153	Application Data

Frame 30: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{C88E6B3E-ACC5-4A9...}

Ethernet II, Src: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61), Dst: Intel_ec:9d:d9 (e4:60:17:ec:9d:d9)

Internet Protocol Version 6, Src: 2606:4700:4403::ac40:94eb, Dst: 2600:1700:3490:7550:4c1f:5809:3f33:6a53

Transmission Control Protocol, Src Port: 443, Dst Port: 52075, Seq: 80, Ack: 181, Len: 0

30	2.276874600	2606:4700:4403...	2600:1700:3490:75...	TCP	74	443 → 52075 [ACK] Seq=80 Ack=181 Win=17 Len=0
35	2.334768500	2606:4700:4403...	2600:1700:3490:75...	TLSv1.2	153	Application Data
40	2.385259800	2600:1700:3490...	2606:4700:4403::a...	TCP	74	52075 → 443 [ACK] Seq=181 Ack=159 Win=252 Len=0

Frame 35: Packet, 153 bytes on wire (1224 bits), 153 bytes captured (1224 bits) on interface \Device\NPF_{C88E6B3E-ACC5-4A9...}

Ethernet II, Src: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61), Dst: Intel_ec:9d:d9 (e4:60:17:ec:9d:d9)

Internet Protocol Version 6, Src: 2606:4700:4403::ac40:94eb, Dst: 2600:1700:3490:7550:4c1f:5809:3f33:6a53

Transmission Control Protocol, Src Port: 443, Dst Port: 52075, Seq: 80, Ack: 181, Len: 79

Transport Layer Security

This is very interesting, so while I specifically looked for TCP protocol, it also provided TLS information. This is because TCP (Transmission control protocol) provides reliable delivery and TLS (transport layer security) encrypts communication, so they often work together. You will see that these use port 443 which is used for secure web traffic (HTTPS). I wonder if any of the sites I went to use port 80, which is for unsecured web traffic (HTTP)

http						
No.	Time	Source	Destination	Protocol	Length	Info
176	3.077845600	192.168.1.165	52.44.109.51	HTTP	343	GET /list.php?v=1 HTTP/1.1
179	3.109683200	52.44.109.51	192.168.1.165	HTTP	874	HTTP/1.1 200 OK (text/html)
23234	28.509467800	2600:1700:3490...	2607:f8b0:4002:c1...	HTTP	285	GET /we1/uCgIcmwvftc.crl HTTP/1.1
23243	28.527389900	2607:f8b0:4002...	2600:1700:3490:75...	PKIX-C...	1110	Certificate Revocation List


```

> Frame 176: Packet, 343 bytes on wire (2744 bits), 343 bytes captured (2744 bits) on interface \Device\NPF_{C88E6
> Ethernet II, Src: Intel_ec:9d:d9 (e4:60:17:ec:9d:d9), Dst: NokiaSolutio_c3:2d:61 (40:48:6e:c3:2d:61)
> Internet Protocol Version 4, Src: 192.168.1.165, Dst: 52.44.109.51
> Transmission Control Protocol, Src Port: 62101, Dst Port: 80, Seq: 1, Ack: 1, Len: 289
> Hypertext Transfer Protocol

```

Oh! I did have some. Neat, so this was traffic that passed through port 80. Now I do see a CRL a few lines down, but why was it pulled via port 80? After researching, I learned that CRLs are commonly distributed over HTTP because the data is public and is used by clients to verify that certificates have not been revoked before establishing trust.

From all this information I can form a timeline of events:

Open Gmail browser >DNS lookup >TLS handshake >CRL check

What surprised me:

- I expected HTTPS traffic to completely replace HTTP, but I discovered Certificate Revocation Lists are still commonly distributed over HTTP because the information is public.
- I was surprised by how many packets were generated from simply opening three websites.